

Backend Developer Guide — Spring Boot API

Read [OVERVIEW.md](#) first if you haven't. This document is the deep dive on `backend/` — architecture, request pipeline, persistence, protocol validation, and the specific bugs we hit while building this and how they were fixed, so you don't have to rediscover them.

1. Tech stack & package layout

- **Java 21, Spring Boot 3.3.6** (Web, Security, Validation, Actuator, JDBC — no Spring Data JPA, we talk to Postgres with plain `JdbcTemplate`)
- **No Lombok**. Plain POJOs with manual getters/setters and `@JsonInclude(NON_NULL)` so optional fields don't clutter JSON responses.
- **Constructor injection** everywhere — no field `@Autowired` .
- Build tool: Maven (`./mvnw` , wrapper committed — you don't need Maven installed globally).

Flat, by-layer packages under `com.yuzee.tokenlab` :

Package	Contents
<code>controller/</code>	REST + SSE endpoints. See § 3 for the full list.
<code>service/</code>	All business logic — the chat pipeline, warehouse, objectives, mini-pathway, whiteboard, research, routing, token accounting.
<code>service/warehouse/</code>	The SQLite/FTS5 course-and-labour-market data pipeline.
<code>protocol/</code>	The "Yuzee Response Protocol" envelope types and <code>ProtocolValidator</code> .
<code>model/</code>	Plain POJOs — <code>Conversation</code> , <code>ChatMessage</code> , <code>ChatRequest</code> , <code>ModelInfo</code> , <code>CompactionMetrics</code> , <code>ObjectiveSession</code> , etc.
<code>repository/</code>	<code>ConversationRepository</code> interface with two implementations (Postgres-backed and file-backed), chosen automatically at boot.
<code>config/</code>	<code>SecurityConfig</code> , <code>CorsConfig</code> , <code>PersistenceConfig</code> , <code>WarehousePlannerWiring</code> .
<code>auth/</code>	<code>HmacTokenFilter</code> — the single-admin bearer-token auth filter.
<code>filter/</code>	<code>RateLimitFilter</code> — in-memory per-endpoint token-bucket limiting.

Resources live under `backend/src/main/resources/` :

- `prompts/` — the real system prompt (`system-prompt.md` , ~3,500 lines), both protocol JSON Schemas (`response-schema-v1.3.json` , `-v1.4.json`), and the mini-pathway prompt/override files.
- `config/` — supporting JSON data: `microtools.json` , `service-registry.json` , `learningContracts.json` , `learningProfiles.json` , `skillInputContracts.json` , `counsellingStyle.json` , `bgeCalibration.json` , `bgeArtifact.json` , and the `objectives/` catalogue (a ~316-item structured-activity catalogue plus its execution-prompt registry).
- `static/` — this is where the compiled Angular build gets copied for a single-jar production deploy (`npm run build:spring` in `frontend/`). It's empty in a fresh checkout; `SpaController` falls back to serving whatever's there (or nothing, in pure-dev-mode where the frontend runs separately on `:4200`).

2. How a chat turn works

This is the core of the whole app: `POST /api/conversations/{id}/messages` , handled by `ChatController.runTurn` , streamed back as Server-Sent Events. In order:

1. `RequestAssemblerService.assembleRequest`

- Runs a deterministic **bypass classifier** first: greetings, farewells, idle chit-chat, and similar low-value turns never touch Gemini at all — a fixed canned response is returned immediately. This is a real cost/latency optimization, not a stub.
- For real turns, calls into `ConversationMemoryService` to decide which prior turns fit the token budget. Three strategies: keep-everything (short conversations), keyword-relevance (pull in only turns that match the current topic), and newest-first eviction (drop the oldest turns once we're over budget).
- Builds the multi-turn Gemini `contents` array via `MultiTurnRequestBuilder` .
- Resolves a per-message "thinking level" from a keyword heuristic (some questions warrant a deeper/slower model pass than others).
- Attaches the sanitized structured-output JSON Schema (Gemini's structured output mode has quirks the schema has to be massaged around — see `RequestAssemblerService#sanitizeSchemaForGemini` and the empty-enum-null handling described in § 4 below).

2. `SystemPromptCacheManager` — looks up (or kicks off in the background) an explicit Gemini context cache for the ~200KB system prompt, so it isn't resent in full on every single turn. Falls back to sending it inline if a cache isn't ready yet.

3. `GeminiService.streamGenerateRich` — streams the model's response over SSE, wrapped by `ProviderRecoveryService` , which allows exactly one retry, and only if no output has reached the client yet. It will never retry a stream that's already partially delivered to the user (that would duplicate/corrupt what they've already seen).

4. `ProtocolValidator` (see § 4) — validates the complete response against the JSON Schema plus a set of hand-ported semantic invariants.

5. `TeachingAnswerReviewService` + `ReviewRetryService` — for substantive answers (roughly: ≥ 120 words or ≥ 3 titled content blocks), runs a bounded second Gemini pass that fact-checks the response against the actual conversation history (a common failure mode: the model turning something the user said in passing, e.g. "I've done some scripting," into an inflated, unsupported skill claim in its reply). If the review pass itself fails for any reason, the original valid answer is used — the review is a quality improvement, never a hard dependency.

6. `SecurityStateService` — **overwrites** the model's own self-reported security-penalty state with the server's authoritative value. Never trust the model to accurately self-report things like a "security breach count" — that has to be tracked and enforced server-side.

7. **Persistence + response** — the turn is persisted via `ConversationService` / `ConversationLogService` (see § 5), priced via `GeminiModelRegistry` (a real 10-model registry with accurate per-model pricing, not a flat estimate), and streamed to the client as a sequence of SSE events: `phase` (status updates like "reviewing"), `chunk` (incremental text), and a final `done` payload containing the fully parsed structured response, token usage, and compaction metrics.

The actual "Yuzee Response Protocol" is a large, strict JSON contract: content blocks (text/list/table/comparison/steps/timeline/chart/etc.), an active interaction (a question with select/form/text input), service-trigger state, RMO ("ready or missing") readiness tracking, and confidence-band tracking. See `response-`

`schema-v1.3.json` for the schema and `system-prompt.md` for the prompt that drives it — that file is long, but it is the actual, real, fully-fleshed-out prompt, not a stand-in.

3. Full endpoint list

Auth (`AuthController` , `/api/auth`) `POST /login` , `POST /logout` , `GET /check`

Conversations (`ConversationController` , `/api/conversations`) `GET /` , `POST /` , `GET /{id}` , `PUT /{id}` , `DELETE /{id}` , `POST /{id}/generate-title` , `POST /{id}/feedback` , `POST /{id}/reset-memory` , `POST /restore`

Chat / turn-level features (`ChatController` , `/api/conversations`) `POST /{id}/messages` (SSE, the core turn), `GET|POST /{id}/mini-pathway` , `GET|POST /{id}/details` (research/"Explore more"), `GET /{id}/objectives` , `POST /{id}/objectives/{operation}` , `POST /{id}/actions/{actionId}/execute` (simulated trusted-service-action execution — always reports `isConnectedInLab:false` , this is intentional, matching the original prototype)

Routing (`RoutingController` , `/api/routing`) — currently **not called from any live chat turn** (see [OVERVIEW.md §5](#)): `GET /skills` , `POST /validate` , `POST /llm`

System / tooling (`SystemController` , various `/api/*` paths) `GET /api/db-status` , `GET /api/protocol/info` , `GET /api/config/capabilities` , `GET|POST /api/system-prompt(/reload)` , `GET|PUT /api/shared-settings(/reset-prompt)` , `GET|POST /api/tokens/session-stats(/session-reset)` , `GET /api/tokens/log` , `GET /api/tokens/lifetime-stats` , `GET /api/tokens/daily-cost` , `GET /api/tokens/utility-stats` (⚠ stub, always zero — see gaps), `POST /api/tokens/count` , `GET /api/warehouse/status` , `GET /api/objectives/catalogue` , `GET /api/pathway/stats` , `POST /api/benchmark` , `POST /api/extract-profile-facts` , `POST /api/detect-contradictions` , `POST /api/pre-check` , `POST /api/pathway/generate|recommend|explain` , `POST /api/conversations/load-demo` (seeds a demo conversation for quick manual testing)

4. Protocol validation — three layers

`ProtocolValidator` (`protocol/ProtocolValidator.java`) is a faithful, tested port of the original prototype's validator, and it's worth understanding because you'll see its output surfaced in the frontend as "⚠ Format validation failed — showing raw output" when a response fails it.

1. **Layer 1 — JSON Schema conformance.** Uses `networknt/json-schema-validator` (draft-07) against `response-schema-v1.3.json` or `-v1.4.json` , plus a few hand-checked envelope-shape requirements (the schema alone doesn't enforce every nested requirement cleanly).
2. **Layer 2 — semantic & invariant rules** hand-ported from the original prompt contract. Examples: the first content block must be plain text with no heading; `ranked_select` interactions need 3–6 options; a `handoff` interaction must use a `fields` input type; confidence score and confidence band must agree (`score 0–39 ⇒ band="low"` , etc.); comparison-table rows must have exactly one cell per declared column.
3. **Layer 3 — trusted service-action registry check.** Any `service_trigger` action the model claims must exist in `TrustedServiceActions` 's hardcoded registry, or it's flagged as untrusted (a warning, not a hard failure — this only matters if/when a real trusted action integration is ever wired in).

There's also a **known, deliberate Gemini quirk workaround**: `normalizeGeminiEmptyEnumNulls()` . Some schema fields (`active_security_penalty` , `status` , `rmo_type`) have `""` as a legal enum value, but Gemini's structured output mode can't represent an empty-string enum member directly — it returns JSON `null` instead. Without this normalization, a perfectly well-formed response with (for example) no active security penalty would fail

schema validation on a pure representational quirk, not a real problem. If you add a new enum field that allows `""`, you'll likely need to add it to `EMPTY_ENUM_NULLABLE_FIELDS` too.

Why does validation ever fail on a well-formed request? Sometimes it doesn't — it's an LLM. Gemini occasionally doesn't perfectly follow the (very long and strict) protocol contract, and when that happens the app is *designed* to fall back to showing the raw JSON rather than crash or silently drop the turn. That's expected, occasional behavior inherent to working with an LLM, not a defect in this port specifically — the same thing happened in the original prototype.

There's a second, narrower validator surface: `validateUserEventAgainstActiveInteraction` and `validateInteractionFields`, which re-validate a client-submitted answer (option pick, form fields, free text) against the **server's own record** of the last interaction it actually sent — never trust a client-echoed copy of "what question was I answering."

5. Persistence

Controlled entirely by whether `DATABASE_URL` is set (`PersistenceConfig`):

- **Set** → Postgres via a manually-built HikariCP `DataSource` + `JdbcTemplate` .
`JdbcConversationRepository` / `JdbcConversationLogService` .
- **Unset** → a local JSON file store under `backend/data/` . `FileConversationRepository` / `FileConversationLogService` . Fine for solo dev, not shared across machines, no real concurrency guarantees.

`TokenlabApplication` explicitly excludes Spring Boot's own `DataSourceAutoConfiguration` / `JdbcTemplateAutoConfiguration` — `PersistenceConfig` is the *only* place a `DataSource` bean gets created, and only when the URL is non-blank. This is deliberate: it means a blank `DATABASE_URL` never triggers Spring's default "I couldn't build a `DataSource`" boot failure.

Two real bugs we hit here (and their fixes)

1. Postgres `jsonb` vs `text` column mismatch. The Postgres schema is a **pre-existing, shared schema** (this app writes to the same Supabase Postgres project the original Node prototype used). One column, `career_context`, is typed `jsonb`; every other JSON-ish column (`profile_facts`, `mini_pathways`, `details`, `objectives`) is plain `text`. A naive `INSERT ... VALUES (?, ?, ...)` failed with:

```
column "career_context" is of type jsonb but expression is of type character varying
```

Fixed with a targeted `?::jsonb` cast **only on that one column** in `JdbcConversationRepository`'s insert SQL — verified against `information_schema.columns` directly, not guessed. If you add new Postgres-persisted JSON columns, check the real column type first; don't assume they're all the same.

2. PgBouncer + JDBC prepared-statement collision. `DATABASE_URL` points at Supabase's connection **pooler** (port `6543`, PgBouncer in *transaction* pooling mode), not a direct Postgres connection. Under load this intermittently threw:

```
org.postgresql.util.PSQLException: ERROR: prepared statement "S_1" already exists
```

Why: pgjdbc's default optimization server-side-prepares statements and names them sequentially (`S_1`, `S_2`, ...) per JDBC connection. In transaction-pooling mode, PgBouncer hands different logical clients whichever physical backend connection is free — so a statement name one client already prepared on that physical connection can collide with a name pgjdbc tries to reuse for a different logical client. This is a well-known PgBouncer/pgjdbc interaction, not specific to this app.

Fixed in `PersistenceConfig#parse()` by appending `&prepareThreshold=0` to the JDBC URL — this disables pgjdbc's server-side prepared-statement optimization entirely (always sends plain/unnamed statements), which is the standard fix for PgBouncer compatibility. Safe for direct (non-pooled) connections too; it just gives up an optimization, never causes incorrect behavior.

3. Duplicate message-ID collisions (demo seeding). The `/api/conversations/load-demo` endpoint used to hardcode message IDs (`user-demo-1` , `asst-demo-1`) — fine for the original file-per-conversation Node store, which tolerated repeated literals across separate files, but a real problem once `messages.id` became a single global Postgres primary key: clicking "Load Demo" a second time threw a `DuplicateKeyException` . Fixed by removing the hardcoded IDs entirely and relying on `ChatMessage` 's default random-UUID constructor.

6. Auth & rate limiting

- **Auth:** `HmacTokenFilter` — a single hardcoded admin account (no user registration/multi-tenancy). On login, a bearer token is issued, HMAC-signed with `auth.secret` (`AUTH_SECRET` env var). **The dev default secret is hardcoded in `application.properties` — set a real `AUTH_SECRET` before any real deployment.**
- **Rate limiting:** `RateLimitFilter` — a simple in-memory per-endpoint token bucket (10–30 requests/min on the LLM-calling endpoints). This resets on restart and isn't shared across multiple backend instances — fine for a single-process deployment, would need a shared store (Redis, etc.) for a horizontally-scaled one.

7. The warehouse subsystem

`service/warehouse/` is a real, working data pipeline over a 288-table SQLite database (`training_gov.db` , ~12GB — Australian government training/course/job market data: `training.gov.au` courses, CRICOS/HE providers, NCVER stats, ABS census data, job-service-Australia projections, O*NET occupation/skill mappings, funding rules, TimesFM-based demand forecasting, and more).

- `WarehouseIndexBuilder` builds an FTS5 (full-text search) index over the raw SQLite tables, with JDBC-stamp-based rebuild avoidance (it only rebuilds the index if the source file's mtime/size changed since the index was last built).
- `WarehouseQueryService` runs the actual course/provider/job/region lookups.
- `WarehousePlanner` (wired via `WarehousePlannerWiring`) is a Gemini-backed query planner — given a natural-language need, it decides what to actually query for.
- `WarehouseService` is the public-facing facade: `status()` , `isAvailable()` , `needsWarehouse()` , `lookup()` , `retrieve()` , `explorationChoice()` , `workspaceCourses()` .
- `RoleRankingService` does lexical (not embedding-based) role/course relevance ranking — a deliberate, documented simplification versus the original prototype's server-side BGE embedding ranking, to avoid pulling an ONNX runtime into the Java backend for one feature.

Current status: wired into the live chat turn. `ChatController.runTurn()` calls `warehouseService.retrieve()` right after the TurnNeeds preflight (advisory only — `needsWarehouse()` cheaply gates out trivial/off-topic turns before any LLM planning call runs), and attaches the resulting `WarehousePack` to the SSE `done` payload as `warehouseData` whenever its status is `READY` or `NO_MATCH` (never for `NOT_NEEDED` / `UNAVAILABLE` , to avoid sending empty noise every turn). Verified end-to-end against the real 12GB data file: real course/provider/career search results and a real "multiple providers match this name" disambiguation, rendered live in chat via `WarehouseCoursesComponent` / `WarehouseConnectionsComponent` / `ProviderComparisonComponent` (see `FRONTEND.md § 4`).

Two real issues surfaced by actually wiring this in (both fixed, worth knowing about if you touch the planner):

1. The very first call after a source-data change can block for a long time.

`WarehouseIndexBuilder.ensureReady()` is `synchronized` and, the first time it runs against a changed/new source file, rebuilds the on-disk FTS5 index — which took about 90 seconds against the real 12GB file in testing. A chat turn must never make a user wait that long for an *advisory* enrichment, so the call is wrapped in a bounded timeout (`ChatController` submits it to the existing turn executor and gives up after 4 seconds if it hasn't returned — see the `warehousePack` block in `runTurn()`). The index build itself keeps running to completion on that executor thread even after the turn stops waiting on it, so later turns benefit from a warm index.

2. The planner call needs a `responseSchema`, not just a prose system instruction.

`WarehousePlannerWiring` 's Gemini call originally used `responseMimeType: application/json` alone — free-form JSON mode. Without an explicit schema, Gemini is only as reliable as the prompt's prose description of field names, and in testing it consistently returned `target` instead of the required `action` key, failing `WarehouseService.retrieve()` 's validation every time. Fixed by adding a `GeminiService.generateJson(..., responseSchema)` overload and building an explicit schema in `WarehousePlannerWiring` that matches `retrieve()` 's field names/enums exactly. One further gotcha while building that schema: Gemini's schema validator rejects `""` as an enum member outright (`400 INVALID_ARGUMENT`) — where the original TS-ported code used an empty string to mean "no value" (e.g. `location.state` when no state was given), the schema instead marks that property `nullable: true` and uses JSON `null` for "not given."

Setup: set `YUZEE_WAREHOUSE_DB` to the SQLite file's path and `WAREHOUSE_INDEX_PATH` to where you want the FTS5 index cached. Without `YUZEE_WAREHOUSE_DB` set, warehouse features cleanly report "unavailable" — this is the expected, safe default, not an error state. **The warehouse database itself is restricted internal data — never commit it to git or place it anywhere public.** It's already covered by the root `.gitignore` 's `backend/data/` rule; keep it that way.

8. Objectives, mini-pathway, whiteboard, research — brief notes

- **Objectives** (`ObjectiveService` , `ObjectiveCatalogueService` , `ObjectiveWorkspacePolicyService`): drives a ~316-item structured-activity catalogue (a guided "pick one thing to work through" workspace, separate from free-form chat). Session state (`ObjectiveSession`) tracks start/advance/ correct/dismiss/list/handoff. No warehouse data reaches this yet (see § 7).
- **Mini-pathway** (`MiniPathwayService` , `PathwayContextService` , `PathwayPresentationService` , `PathwayBlockStreamParser`): generates a short AI-written career-route report, streamed incrementally (the parser handles partial/incomplete JSON arrays as they stream in, rather than waiting for the whole response).
- **Pathway Whiteboard** (`PathwayWhiteboardService` , `PathwayPolicyService`): the larger drag-and-drop pathway editor's backend — `generate / recommend / explain` endpoints. `PathwayPolicyService` 's auto-trigger confidence gate (deciding *when* to proactively offer a mini-pathway) is currently dead code — nothing calls it, so mini-pathway generation is manual-trigger-only in this build.
- **Research / "Explore more"** (`DetailResearchService`): a two-stage Gemini grounded-search flow with an SSRF-safe URL allowlist (`SafeUrlValidator`) for source links. Fully wired and live — `ChatController` computes a `researchOffer` on every turn via `TurnNeedsService` , and the frontend surfaces it as an "Explore more" card.

9. Testing & local dev workflow

```
cd backend
./mvnw test           # JUnit, runs against the file-store fallback (no real DB
```

```
needed)
./mvnw spring-boot:run           # full app; needs GEMINI_API_KEY at minimum
```

A few things worth knowing when running locally on Windows:

- If you see `WARNING: A restricted method in java.lang.System has been called` at boot — that's just JDK 21+'s native-access warnings from Maven/Tomcat/SQLite JNI libraries. Harmless, not a real error.
- If `mvnw.cmd` seems to silently do nothing under a plain `cmd.exe` invocation, make sure `JAVA_HOME` is actually exported in that shell first — an uninitialized shell (e.g. a fresh background process) won't inherit it automatically.
- Compile-only check (fast, skips tests): `./mvnw -q compile` .

10. What's genuinely stubbed vs. real

Everything under `service/` produces real logic against real data — this was verified file-by-file, not assumed from directory names. The **only** confirmed stub: `GET /api/tokens/utility-stats` always returns `{whiteboardCalls: 0, utilityModelCalls: 0}` — the counter it's meant to read was never wired up (there's a comment to this effect in `PathwayWhiteboardService`). Low risk (it's a telemetry endpoint), but flagged here so nobody spends time debugging "why is this always zero."